



Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

## **Actividad 7: Procesamiento secuencial y multihilo**

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 24 de septiembre de 2026

# Índice general

|                                                                                            |           |
|--------------------------------------------------------------------------------------------|-----------|
| Índice de figuras                                                                          | III       |
| Índice de cuadros                                                                          | IV        |
| <b>1. Comparación secuencial y multihilo</b>                                               | <b>1</b>  |
| 1.1. Objetivo . . . . .                                                                    | 1         |
| 1.2. Entradas . . . . .                                                                    | 1         |
| 1.3. Procesamiento . . . . .                                                               | 2         |
| 1.4. Estructuras de control utilizadas . . . . .                                           | 2         |
| 1.5. Medición del tiempo . . . . .                                                         | 3         |
| 1.6. Comprobación de resultados . . . . .                                                  | 3         |
| 1.7. Resultados . . . . .                                                                  | 3         |
| <b>2. Análisis y conclusiones</b>                                                          | <b>10</b> |
| 2.1. ¿En qué casos fue más rápida cada implementación? . . . . .                           | 10        |
| 2.2. ¿Cómo influyeron el tamaño del arreglo y la complejidad de<br>la operación? . . . . . | 10        |
| 2.3. ¿Por qué utilizar varios hilos no siempre mejora el rendimiento?                      | 11        |
| 2.4. Conclusión . . . . .                                                                  | 11        |
| <b>Referencias</b>                                                                         | <b>12</b> |

# Índice de figuras

|                                                                                              |   |
|----------------------------------------------------------------------------------------------|---|
| 1.1. Código Java, parte 1 de 4: generación de los arreglos y suma<br>secuencial. . . . .     | 5 |
| 1.2. Código Java, parte 2 de 4: suma multihilo y división del arreglo. . . . .               | 6 |
| 1.3. Código Java, parte 3 de 4: procesamiento de mayor complejidad con cuatro hilos. . . . . | 7 |
| 1.4. Código Java, parte 4 de 4: medición de tiempos y ejecución de las pruebas. . . . .      | 8 |
| 1.5. Salida real del programa con los tiempos promedio. . . . .                              | 9 |

# Índice de cuadros

|                                                                  |   |
|------------------------------------------------------------------|---|
| 1.1. Comparación de tiempos promedio entre las implementaciones. | 4 |
|------------------------------------------------------------------|---|

# 1. Comparación secuencial y multihilo

Comparar el rendimiento de una implementación secuencial y una multihilo según el tamaño y la complejidad del problema. Se generan arreglos de enteros entre 1 y 100 con 40, 100,000, 1,000,000 y 10,000,000 de elementos. Para cada arreglo se realizan una suma sencilla y una operación de mayor procesamiento,

$$\sqrt{x} \log(x + 1),$$

tanto de manera secuencial como con cuatro hilos. Cada implementación se ejecuta diez veces y se registra el tiempo promedio sin incluir la generación del arreglo.

## 1.1 Objetivo

El objetivo es observar cómo influyen dos variables sobre el tiempo de ejecución: el número de elementos del arreglo y la cantidad de cómputo realizada por elemento. La comparación permite identificar cuándo el costo de crear, iniciar, planificar y sincronizar cuatro hilos puede ser compensado por el trabajo que se ejecuta de forma concurrente.

## 1.2 Entradas

El programa utiliza cuatro tamaños de arreglo:

40,      100 000,      1 000 000,      10 000 000.

Cada posición contiene un entero pseudoaleatorio entre 1 y 100. El mismo arreglo se utiliza para comparar las dos implementaciones de una operación, de manera que ambas reciben exactamente los mismos datos.

### 1.3 Procesamiento

Para la prueba de suma sencilla, la versión secuencial recorre el arreglo completo con un solo flujo y acumula los valores en una variable `long`. La versión multihilo divide los índices del arreglo en cuatro intervalos. Cada hilo calcula un subtotal y lo guarda en una posición exclusiva de un arreglo de resultados parciales.

Para la prueba de mayor procesamiento se utiliza, para cada elemento  $x$ ,

$$f(x) = \sqrt{x} \log(x + 1).$$

En Java se implementa mediante `Math.sqrt(valor) * Math.log(valor + 1.0)`. La versión secuencial acumula todos los valores en un único flujo; la versión multihilo aplica la misma división en cuatro intervalos y calcula cuatro subtotales.

Los cuatro hilos se crean primero y se inician mediante `start()`. Posteriormente, el hilo principal ejecuta `join()` sobre cada uno. Solo después de que los cuatro han terminado se combinan sus resultados parciales.

### 1.4 Estructuras de control utilizadas

Se emplean ciclos `for` para generar los datos, recorrer los arreglos y repetir cada medición diez veces. La versión multihilo utiliza un arreglo de cuatro objetos `Thread`. Cada hilo recibe una tarea mediante una expresión lambda y procesa el intervalo

$$\left[ \frac{in}{4}, \frac{(i+1)n}{4} \right),$$

donde  $i \in \{0, 1, 2, 3\}$  es el identificador del hilo y  $n$  es el tamaño del arreglo.

Cada hilo escribe únicamente en su propia posición del arreglo de resultados parciales. Por ello no se requiere un cerrojo para los subtotales.

Las llamadas a `join()` garantizan que todos los resultados parciales estén disponibles antes de combinarlos.

## 1.5 Medición del tiempo

La generación del arreglo se realiza antes de iniciar cualquier cronómetro, tal como solicita la actividad. Cada implementación se ejecuta diez veces. Para cada repetición se utiliza `System.nanoTime()` y se acumula la duración. El promedio en milisegundos se calcula como

$$\bar{t} = \frac{\sum_{k=1}^{10} t_k}{10 \cdot 10^6}.$$

El tiempo de la versión multihilo incluye la creación, el inicio y la sincronización de los hilos, pues esas operaciones forman parte de la implementación que se desea comparar.

## 1.6 Comprobación de resultados

Para la suma sencilla, los resultados secuencial y multihilo deben coincidir exactamente. Para la operación con `double`, el orden de las sumas cambia entre ambas implementaciones y puede generar diferencias mínimas de redondeo. Por ello se comprueba equivalencia con una tolerancia relativa de  $10^{-10}$ . Si alguna comprobación falla, el programa lanza una excepción en lugar de presentar tiempos de implementaciones que producen resultados distintos.

## 1.7 Resultados

Los tiempos son dependientes del procesador, de la JVM y de la carga del sistema. El Cuadro 1.1 presenta los promedios obtenidos en la última ejecución visible en la evidencia de la computadora utilizada. Cada valor corresponde al promedio de diez repeticiones del programa.

## Ejercicio 1. Comparación secuencial y multihilo

**Cuadro 1.1:** Comparación de tiempos promedio entre las implementaciones.

| Tamaño     | Operación           | Secuencial (ms) | Multihilo (ms) | Más rápido |
|------------|---------------------|-----------------|----------------|------------|
| 40         | Suma                | 0.000           | 0.256          | Secuencial |
| 40         | Mayor procesamiento | 0.011           | 0.247          | Secuencial |
| 100,000    | Suma                | 0.217           | 0.375          | Secuencial |
| 100,000    | Mayor procesamiento | 0.642           | 0.930          | Secuencial |
| 1,000,000  | Suma                | 0.292           | 0.289          | Multihilo  |
| 1,000,000  | Mayor procesamiento | 5.003           | 2.192          | Multihilo  |
| 10,000,000 | Suma                | 2.518           | 1.065          | Multihilo  |
| 10,000,000 | Mayor procesamiento | 44.029          | 20.945         | Multihilo  |

## Ejercicio 1. Comparación secuencial y multihilo

```
J Ejercicio01ProcesamientoSecuencialMultihilo.java M X
07 Procesamiento secuencial y multihilo > code > java > J Ejercicio01ProcesamientoSecuencialMultihilo.java >
1  import java.util.Random;
2
3  public class Ejercicio01ProcesamientoSecuencialMultihilo {
4      // configura la cantidad de hilos y repeticiones de la actividad
5      private static final int HILOS = 4;
6      private static final int REPETICIONES = 10;
7      private static final int[] TAMANOS = {40, 100_000, 1_000_000, 10_000_000};
8
9      // usa una semilla fija para que los arreglos sean reproducibles
10     private static final Random ALEATORIO = new Random(seed: 2026L);
11
12     // genera valores enteros entre 1 y 100 fuera de la medicion
13     private static int[] generarArreglo(int tamano) {
14         int[] datos = new int[tamano];
15
16         for (int i = 0; i < datos.length; i++) {
17             datos[i] = 1 + ALEATORIO.nextInt(bound: 100);
18         }
19
20         return datos;
21     }
22
23     // acumula todo el arreglo en un solo flujo de ejecucion
24     private static long sumaSecuencial(int[] datos) {
25         long suma = 0;
26
27         for (int valor : datos) {
28             suma += valor;
29         }
30
31         return suma;
32     }
}
```

Figura 1.1: Código Java, parte 1 de 4: generación de los arreglos y suma secuencial.

## Ejercicio 1. Comparación secuencial y multihilo

```
J Ejercicio01ProcesamientoSecuencialMultihilo.java M X
07 Procesamiento secuencial y multihilo > code > java > J Ejercicio01ProcesamientoSecuencialMultihilo.java >
3  public class Ejercicio01ProcesamientoSecuencialMultihilo {
34      // divide el arreglo en cuatro secciones y suma cada una en un hilo
35      private static long sumaMultihilo(int[] datos) throws InterruptedException {
36          Thread[] hilos = new Thread[HILOS];
37          long[] parciales = new long[HILOS];
38
39          for (int id = 0; id < HILOS; id++) {
40              int idFinal = id;
41              int inicio = id * datos.length / HILOS;
42              int fin = (id + 1) * datos.length / HILOS;
43
44              hilos[id] = new Thread(() -> {
45                  long subtotal = 0;
46
47                  for (int i = inicio; i < fin; i++) {
48                      subtotal += datos[i];
49                  }
50
51                  parciales[idFinal] = subtotal;
52              }, "Suma-" + id);
53          }
54
55          // start inicia los cuatro hilos
56          for (Thread hilo : hilos) {
57              hilo.start();
58          }
59
60          // join espera a que todos terminen antes de combinar resultados
61          for (Thread hilo : hilos) {
62              hilo.join();
63          }
64
65          long total = 0;
66
67          for (long parcial : parciales) {
68              total += parcial;
69          }
70
71          return total;
72      }
73
74      // calcula sqrt(x) por log(x + 1) sobre todo el arreglo
75      private static double mayorProcesamientoSecuencial(int[] datos) {
76          double acumulado = 0.0;
77
78          for (int valor : datos) {
79              acumulado += Math.sqrt(valor) * Math.log(valor + 1.0);
80          }
81
82          return acumulado;
83      }
84  }
```

Figura 1.2: Código Java, parte 2 de 4: suma multihilo y división del arreglo.

## Ejercicio 1. Comparación secuencial y multihilo

```
J Ejercicio01ProcesamientoSecuencialMultihilo.java M X
07 Procesamiento secuencial y multihilo > code > java > J Ejercicio01ProcesamientoSecuencialMultihilo.java >
3 public class Ejercicio01ProcesamientoSecuencialMultihilo {
85 // reparte el procesamiento costoso entre cuatro hilos
86 private static double mayorProcesamientoMultihilo(int[] datos)
87     throws InterruptedException {
88     Thread[] hilos = new Thread[HILOS];
89     double[] parciales = new double[HILOS];
90
91     for (int id = 0; id < HILOS; id++) {
92         int idFinal = id;
93         int inicio = id * datos.length / HILOS;
94         int fin = (id + 1) * datos.length / HILOS;
95
96         hilos[id] = new Thread(() -> {
97             double subtotal = 0.0;
98
99             for (int i = inicio; i < fin; i++) {
100                 int valor = datos[i];
101                 subtotal += Math.sqrt(valor) * Math.log(valor + 1.0);
102             }
103
104             parciales[idFinal] = subtotal;
105             }, "Procesamiento-" + id);
106     }
107
108     for (Thread hilo : hilos) {
109         hilo.start();
110     }
111
112     for (Thread hilo : hilos) {
113         hilo.join();
114     }
115
116     double total = 0.0;
117
118     for (double parcial : parciales) {
119         total += parcial;
120     }
121
122     return total;
123 }
124
125 // compara resultados double permitiendo una diferencia numerica muy pequena
126 private static boolean casiIguales(double a, double b) {
127     double escala = Math.max(a: 1.0, Math.max(Math.abs(a), Math.abs(b)));
128     return Math.abs(a - b) <= 1e-10 * escala;
129 }
130
131 // imprime una fila con los promedios de las diez ejecuciones
132 private static void imprimirFila(int tamano, String operacion,
133     double tiempoSecuencialMs, double tiempoMultihiloMs) {
134     String masRapido = tiempoSecuencialMs <= tiempoMultihiloMs
135         ? "Secuencial"
136         : "Multihilo";
137
138     System.out.printf(format: "%,12d | %-20s | %15.3f | %15.3f | %-10s%n",
139         tamano, operacion, tiempoSecuencialMs, tiempoMultihiloMs, masRapido);
140 }
141 }
```

Figura 1.3: Código Java, parte 3 de 4: procesamiento de mayor complejidad con cuatro hilos.

## Ejercicio 1. Comparación secuencial y multihilo

```
J Ejercicio01ProcesamientoSecuencialMultihilo.java M X
07 Procesamiento secuencial y multihilo > code > java > J Ejercicio01ProcesamientoSecuencialMultihilo.java >
3 public class Ejercicio01ProcesamientoSecuencialMultihilo {
142 public static void main(String[] args) throws InterruptedException {
154     long tiempoSumaSecuencial = 0L;
155     long resultadoSumaSecuencial = 0L;
156
157     for (int repeticion = 0; repeticion < REPETICIONES; repeticion++) {
158         long inicio = System.nanoTime();
159         resultadoSumaSecuencial = sumaSecuencial(datos);
160         tiempoSumaSecuencial += System.nanoTime() - inicio;
161     }
162
163     long tiempoSumaMultihilo = 0L;
164     long resultadoSumaMultihilo = 0L;
165
166     for (int repeticion = 0; repeticion < REPETICIONES; repeticion++) {
167         long inicio = System.nanoTime();
168         resultadoSumaMultihilo = sumaMultihilo(datos);
169         tiempoSumaMultihilo += System.nanoTime() - inicio;
170     }
171
172     if (resultadoSumaSecuencial != resultadoSumaMultihilo) {
173         throw new IllegalStateException(
174             s: "La suma secuencial y multihilo no coinciden");
175     }
176
177     double promedioSumaSecuencial = tiempoSumaSecuencial
178         / (double) REPETICIONES / 1_000_000.0;
179     double promedioSumaMultihilo = tiempoSumaMultihilo
180         / (double) REPETICIONES / 1_000_000.0;
181
182     imprimirFila(tamano, operacion: "Suma",
183         promedioSumaSecuencial, promedioSumaMultihilo);
184
185     long tiempoMayorSecuencial = 0L;
186     double resultadoMayorSecuencial = 0.0;
187
188     for (int repeticion = 0; repeticion < REPETICIONES; repeticion++) {
189         long inicio = System.nanoTime();
190         resultadoMayorSecuencial = mayorProcesamientoSecuencial(datos);
191         tiempoMayorSecuencial += System.nanoTime() - inicio;
192     }
193
194     long tiempoMayorMultihilo = 0L;
195     double resultadoMayorMultihilo = 0.0;
196
197     for (int repeticion = 0; repeticion < REPETICIONES; repeticion++) {
198         long inicio = System.nanoTime();
199         resultadoMayorMultihilo = mayorProcesamientoMultihilo(datos);
200         tiempoMayorMultihilo += System.nanoTime() - inicio;
201     }
202
203     if (!casiIguales(resultadoMayorSecuencial, resultadoMayorMultihilo)) {
204         throw new IllegalStateException(
205             s: "El procesamiento secuencial y multihilo no coinciden");
206     }
207
208     double promedioMayorSecuencial = tiempoMayorSecuencial
209         / (double) REPETICIONES / 1_000_000.0;
210     double promedioMayorMultihilo = tiempoMayorMultihilo
211         / (double) REPETICIONES / 1_000_000.0;
212 }
```

Figura 1.4: Código Java, parte 4 de 4: medición de tiempos y ejecución de las pruebas.

## Ejercicio 1. Comparación secuencial y multihilo

```
heri ~/07 Procesamiento secuencial y multihilo/code/java main !? v26.0.2 base 08:48
java Ejercicio01ProcesamientoSecuencialMultihilo.java
Comparacion de procesamiento secuencial y multihilo
Los tiempos no incluyen la generacion del arreglo.
Tamano | Operacion | Secuencial (ms) | Multihilo (ms) | Mas rapido
-----+-----+-----+-----+-----
40 | Suma | 0.001 | 0.285 | Secuencial
40 | Mayor procesamiento | 0.010 | 0.216 | Secuencial
100,000 | Suma | 0.273 | 0.415 | Secuencial
100,000 | Mayor procesamiento | 0.997 | 0.984 | Multihilo
1,000,000 | Suma | 0.402 | 0.254 | Multihilo
1,000,000 | Mayor procesamiento | 5.651 | 3.579 | Multihilo
10,000,000 | Suma | 3.437 | 1.500 | Multihilo
10,000,000 | Mayor procesamiento | 52.250 | 23.220 | Multihilo

heri ~/07 Procesamiento secuencial y multihilo/code/java main !? v26.0.2 base 08:48
java Ejercicio01ProcesamientoSecuencialMultihilo.java
Comparacion de procesamiento secuencial y multihilo
Los tiempos no incluyen la generacion del arreglo.
Tamano | Operacion | Secuencial (ms) | Multihilo (ms) | Mas rapido
-----+-----+-----+-----+-----
40 | Suma | 0.000 | 0.318 | Secuencial
40 | Mayor procesamiento | 0.006 | 0.227 | Secuencial
100,000 | Suma | 0.237 | 0.388 | Secuencial
100,000 | Mayor procesamiento | 0.788 | 0.873 | Secuencial
1,000,000 | Suma | 0.353 | 0.345 | Multihilo
1,000,000 | Mayor procesamiento | 4.980 | 2.085 | Multihilo
10,000,000 | Suma | 2.555 | 1.044 | Multihilo
10,000,000 | Mayor procesamiento | 44.340 | 21.218 | Multihilo

heri ~/07 Procesamiento secuencial y multihilo/code/java main !? v26.0.2 base 08:49
java Ejercicio01ProcesamientoSecuencialMultihilo.java
Comparacion de procesamiento secuencial y multihilo
Los tiempos no incluyen la generacion del arreglo.
Tamano | Operacion | Secuencial (ms) | Multihilo (ms) | Mas rapido
-----+-----+-----+-----+-----
40 | Suma | 0.000 | 0.256 | Secuencial
40 | Mayor procesamiento | 0.011 | 0.247 | Secuencial
100,000 | Suma | 0.217 | 0.375 | Secuencial
100,000 | Mayor procesamiento | 0.642 | 0.930 | Secuencial
1,000,000 | Suma | 0.292 | 0.289 | Multihilo
1,000,000 | Mayor procesamiento | 5.003 | 2.192 | Multihilo
10,000,000 | Suma | 2.518 | 1.065 | Multihilo
10,000,000 | Mayor procesamiento | 44.029 | 20.945 | Multihilo

heri ~/07 Procesamiento secuencial y multihilo/code/java main !? v26.0.2 base 08:49
```

Figura 1.5: Salida real del programa con los tiempos promedio.

## 2. Análisis y conclusiones

### 2.1 ¿En qué casos fue más rápida cada implementación?

La respuesta definitiva debe basarse en los valores medidos de la Table 1.1. Para arreglos pequeños, el trabajo útil puede ser tan breve que el costo de crear, iniciar y sincronizar cuatro hilos sea comparable o incluso mayor que el propio cálculo. En ese caso es razonable observar un menor tiempo en la implementación secuencial.

Al crecer el arreglo, especialmente en la operación  $\sqrt{x} \log(x + 1)$ , existe más cómputo independiente que repartir entre los cuatro hilos. Si la computadora dispone de capacidad de ejecución paralela, la versión multihilo puede entonces reducir el tiempo promedio. Esta interpretación debe contrastarse con los datos obtenidos en la ejecución local.

### 2.2 ¿Cómo influyeron el tamaño del arreglo y la complejidad de la operación?

Aumentar el tamaño incrementa el número de elementos que deben procesarse. Además, la suma sencilla realiza una operación aritmética muy económica, mientras que la segunda prueba evalúa una raíz cuadrada y un logaritmo para cada elemento. Por tanto, el segundo caso asigna más trabajo computacional a cada sección del arreglo.

Cuando el trabajo por elemento es pequeño, los costos asociados con los hilos representan una fracción importante del tiempo total. Al incrementar el tamaño o la complejidad, esa fracción puede disminuir porque cada hilo permanece más tiempo realizando trabajo útil.

## 2.3 ¿Por qué utilizar varios hilos no siempre mejora el rendimiento?

Crear hilos y coordinarlos tiene un costo. Además, el sistema operativo debe planificar su ejecución y el procesador dispone de una cantidad finita de núcleos y recursos compartidos. Si la tarea es demasiado pequeña, el trabajo adicional de administrar los hilos puede superar el tiempo que se pretende ahorrar. Por ello, el uso de varios hilos no implica por sí solo una mejora de rendimiento; el beneficio depende del tamaño del problema, de la complejidad del cálculo y del hardware donde se ejecuta.

## 2.4 Conclusión

La actividad compara dos formas de ejecutar exactamente el mismo cálculo. La versión secuencial evita el costo de coordinación, mientras que la versión multihilo divide el trabajo en cuatro secciones independientes y sincroniza su terminación mediante `join()`. La comparación correcta requiere medir ambas alternativas sobre los mismos datos, excluir la generación del arreglo del cronómetro y verificar que los resultados numéricos sean equivalentes.

Los datos de la [Table 1.1](#) permiten determinar experimentalmente en qué punto el trabajo disponible es suficiente para compensar el costo de usar cuatro hilos.

# Bibliografía

- [1] V. G. Morales Murillo, “Actividad 7: Procesamiento secuencial y multihilo,” 2026, material del curso Programación Concurrente, Universidad de las Américas Puebla.
- [2] W. Stallings, *Operating Systems: Internals and Design Principles*, 7th ed. Boston, MA: Prentice Hall, 2011.